Q: Why doesn't this chapter discuss screen control: moving the cursor, changing the colors of characters on the screen, and so on?

A: C provides no standard functions for screen control. The C standard addresses only issues that can reasonably be standardized across a wide range of computers and operating systems; screen control is outside this realm. The customary way to solve this problem in UNIX is to use the `curses` library, which supports screen control in a terminal-independent manner.

Similarly, there are no standard functions for building programs with a graphical user interface. However, you can most likely use C function calls to access the windowing API (application programming interface) for your operating system.

Exercises

Section 22.1

1. Indicate whether each of the following files is more likely to contain text data or binary data:
   (a) A file of object code produced by a C compiler
   (b) A program listing produced by a C compiler
   (c) An email message sent from one computer to another
   (d) A file containing a graphics image

Section 22.2

2. Indicate which mode string is most likely to be passed to `fopen` in each of the following situations:
   (a) A database management system opens a file containing records to be updated.
   (b) A mail program opens a file of saved messages so that it can add additional messages to the end.
   (c) A graphics program opens a file containing a picture to be displayed on the screen.
   (d) An operating system command interpreter opens a "shell script" (or "batch file") containing commands to be executed.

3. Find the error in the following program fragment and show how to fix it.

```
FILE *fp;

if (fp = fopen(filename, "r")) {
  read characters until end-of-file
}
fclose(fp);
```

Section 22.3

4. Show how each of the following numbers will look if displayed by `printf` with `%#012.5g` as the conversion specification:
   (a) 83.7361
   (b) 29748.6607
   (c) 1054932234.0
   (d) 0.0000235218

5. Is there any difference between the `printf` conversion specifications `%.4d` and `%04d`? If so, explain what it is.

W *6. Write a call of `printf` that prints

`1 widget`

if the `widget` variable (of type `int`) has the value 1, and

n `widgets`

otherwise, where *n* is the value of `widget`. You are not allowed to use the `if` statement or any other statement; the answer must be a single call of `printf`.

*7. Suppose that we call `scanf` as follows:

```
n = scanf("%d%f%d", &i, &x, &j);
```

(`i`, `j`, and `n` are `int` variables and `x` is a `float` variable.) Assuming that the input stream contains the characters shown, give the values of `i`, `j`, `n`, and `x` after the call. In addition, indicate which characters were consumed by the call.

(a) `10•20•30¤`
(b) `1.0•2.0•3.0¤`
(c) `0.1•0.2•0.3¤`
(d) `.1•.2•.3¤`

W 8. In previous chapters, we've used the `scanf` format string `" %c"` when we wanted to skip white-space characters and read a nonblank character. Some programmers use `"%1s"` instead. Are the two techniques equivalent? If not, what are the differences?

Section 22.4

9. Which one of the following calls is *not* a valid way of reading one character from the standard input stream?

(a) `getch()`
(b) `getchar()`
(c) `getc(stdin)`
(d) `fgetc(stdin)`

W 10. The `fcopy.c` program has one minor flaw: it doesn't check for errors as it's writing to the destination file. Errors during writing are rare, but do occasionally occur (the disk might become full, for example). Show how to add the missing error check to the program, assuming that we want it to display a message and terminate immediately if an error occurs.

11. The following loop appears in the `fcopy.c` program:

```
while ((ch = getc(source_fp)) != EOF)
  putc(ch, dest_fp);
```

Suppose that we neglected to put parentheses around `ch = getc(source_fp)`:

```
while (ch = getc(source_fp) != EOF)
  putc(ch, dest_fp);
```

Would the program compile without an error? If so, what would the program do when it's run?

12. Find the error in the following function and show how to fix it.

```
int count_periods(const char *filename)
{
  FILE *fp;
  int n = 0;
```

```
    if ((fp = fopen(filename, "r")) != NULL) {
      while (fgetc(fp) != EOF)
        if (fgetc(fp) == '.')
          n++;
      fclose(fp);
    }

    return n;
}
```

13. Write the following function:

```
int line_length(const char *filename, int n);
```

The function should return the length of line `n` in the text file whose name is `filename` (assuming that the first line in the file is line 1). If the line doesn't exist, the function should return 0.

Section 22.5

W 14. (a) Write your own version of the `fgets` function. Make it behave as much like the real `fgets` function as possible; in particular, make sure that it has the proper return value. To avoid conflicts with the standard library, don't name your function `fgets`.

(b) Write your own version of `fputs`, following the same rules as in part (a).

Section 22.7

W 15. Write calls of `fseek` that perform the following file-positioning operations on a binary file whose data is arranged in 64-byte "records." Use `fp` as the file pointer in each case.

(a) Move to the beginning of record `n`. (Assume that the first record in the file is record 0.)
(b) Move to the beginning of the last record in the file.
(c) Move forward one record.
(d) Move backward two records.

Section 22.8

16. Assume that `str` is a string that contains a "sales rank" immediately preceded by the # symbol (other characters may precede the # and/or follow the sales rank). A sales rank is a series of decimal digits possibly containing commas, such as the following examples:

```
989
24,675
1,162,620
```

Write a call of `sscanf` that extracts the sales rank (but not the # symbol) and stores it in a string variable named `sales_rank`.

Programming Projects

1. Extend the `canopen.c` program of Section 22.2 so that the user may put any number of file names on the command line:

```
canopen foo bar baz
```

The program should print a separate `can be opened` or `can't be opened` message for each file. Have the program terminate with status `EXIT_FAILURE` if one or more of the files can't be opened.

W 2. Write a program that converts all letters in a file to upper case. (Characters other than letters shouldn't be changed.) The program should obtain the file name from the command line and write its output to `stdout`.